

IA Generativa

Módulo 5

Temario

- Definición
- Tecnologías actuales
- Ingeniería de prompts para mejorar respuestas
- LLMs + Langchain

Definición de IA generativa

Se refiere a la Inteligencia Artificial capaz de crear **contenido nuevo** (texto, imágenes, audio, código, etc.).

Tecnologías

Categoría	Modelo(s) o API Más Popular
Modelos de Lenguaje	GPT-4o / GPT-4 (OpenAI), Claude 3 (Anthropic), Gemini (Google), Llama (Meta)
Generación de Imágenes	DALL·E 3 (OpenAI), Midjourney, Stable Diffusion XL (SDXL)
Generación de Código	GitHub Copilot (Basado en GPT), Code Llama (Meta)
Generación de Video/Audio	Sora (OpenAI), Pika, ElevenLabs (Audio), Suno (Música)

Trabajando con LLMs

Definición de LLM

Un **LLM** (por sus siglas en inglés, **Large Language Model** o **Modelo de Lenguaje Grande**) es un tipo de **algoritmo de inteligencia artificial** que ha sido entrenado en cantidades masivas de datos de texto.

El propósito principal de un LLM es **entender, generar y responder** al lenguaje humano de una manera coherente y contextualmente relevante.

- Entrenamiento: Se entrenan utilizando técnicas de aprendizaje profundo en un corpus de texto gigantesco (páginas web, libros, artículos, etc.).
- Capacidad: Debido a su escala y la cantidad de datos que procesan, los LLM son capaces de **aprender patrones lingüísticos complejos, gramática, hechos y una amplia gama de temas**.
- Función: Una vez entrenados, pueden realizar diversas tareas relacionadas con el lenguaje: generar texto, traducir idiomas, resumir documentos, etc.

Ingeniería de Prompts

Definición

La **Ingeniería de Prompts** se define como el arte y la ciencia de **diseñar, refinar y optimizar las instrucciones (o *prompts*)** dadas a los modelos de inteligencia artificial generativa (como GPT, DALL·E, Stable Diffusion) con el objetivo de obtener **la salida más precisa, relevante y deseada**.

Framework Básico: el RTF

- **Rol:** Define la identidad o perspectiva que debe asumir la IA.
- **Tarea:** Describe la acción específica que debe realizar el modelo.
- **Formato:** Indica cómo debe estructurarse la salida (el resultado).

Ejemplo:

- Actúa como un **Analista de Experiencia del Cliente** que se especializa en la minería de texto. Analiza la siguiente lista de comentarios de clientes sobre nuestro nuevo servicio de soporte: **[Insertar lista de comentarios]**.
- Identifica los **dos principales puntos de dolor y el tema positivo más recurrente**.
- Presenta los hallazgos en un **resumen ejecutivo conciso** de no más de 150 palabras, seguido de una sección de "**Recomendación Prioritaria**" con una única viñeta.

Características Universales de un Prompt Efectivo

Un *prompt* efectivo no es una pregunta, es una **especificación de requisitos**.

1) Claridad y Especificidad (Evitar la Ambigüedad)

Este es el principio fundamental. El modelo de IA no puede adivinar tu intención.

- **Claridad:** Utiliza un lenguaje directo y sin errores gramaticales. Evita el doble sentido y las construcciones vagas.
- **Especificidad:** En lugar de decir "Escribe sobre marketing", di: "**Escribe un artículo de 500 palabras sobre marketing de contenidos para pequeñas empresas, centrándote en el SEO local.**"
- **Verbos de Acción:** Usa verbos fuertes y directos que definan la **Tarea** (ej. *Analiza, Compara, Sintetiza, Crea, Resume*).

Características Universales del Prompt Efectivo

2) Contexto Relevante y Rol Asignado

Un modelo sin contexto es un modelo que opera en el vacío; un modelo con un rol opera con un objetivo y una perspectiva definidos.

- **Contexto (Por qué):** Proporciona la información de fondo que la IA necesita para entender la situación. Incluye la **audiencia objetivo** (¿Para quién es esto?) y el **objetivo final** (¿Para qué se usará?).
- **Rol (Quién):** Asigna una **persona** al modelo.
 - *Ejemplo:* "Actúa como un experto en Python con 10 años de experiencia..."
o "Responde como un historiador que enseña a estudiantes de secundaria."

Características Universales del Prompt Efectivo

3) Requisitos de Formato y Estructura (El Resultado)

Si no especificas la forma, el modelo elegirá la forma más común, que quizás no sea la que necesitas.

- **Formato de Salida:** Define exactamente cómo debe presentarse la respuesta:
 - *Ejemplos:* "Responde en una tabla Markdown de tres columnas," "Utiliza una lista numerada," "Elabora un resumen ejecutivo," o "Genera solo el código sin explicaciones introductorias."
- **Restricciones:** Incluye límites de longitud o estilo: "**Máximo 200 palabras**," "**Utiliza un tono persuasivo**," o "**No incluyas emojis**."

Características Universales del Prompt Efectivo

4) Guía Secuencial o Demostración (Pensamiento Estructurado)

Para tareas complejas, el modelo necesita instrucciones sobre cómo pensar.

- **Cadena de Pensamiento (CoT):** Para problemas de razonamiento o lógica, instruye al modelo para que **divida la tarea en pasos lógicos**.
 - *Frase clave:* "Antes de dar la respuesta final, muestra tu razonamiento paso a paso."
- **Few-Shot Prompting:** Si la tarea requiere un estilo o formato único (ej. un tipo específico de email), proporciona **1-3 ejemplos** de la entrada y la salida deseada para "entrenar" al modelo al momento.

Características Universales del Prompt Efectivo

5) Control del Tono y la Personalidad

Esta característica define el **estilo, la actitud y el nivel de emoción** con el que se presenta la información. El control del Tono y la Personalidad asegura que la salida no solo sea correcta, sino también **apropiada para la audiencia y el propósito** del mensaje.

- **Tono:** Definir el sentimiento de la escritura. Ej: "Utiliza un tono motivacional y optimista." O "Mantén un tono formal y académico."
- **Personalidad:** Definir el nivel de entusiasmo y cortesía.
- **Nivel de Lectura:** Ajustar la complejidad del lenguaje. "Escribe esto para un público sin conocimientos técnicos."

Frameworks para Imágenes

Sujeto + Estilo + Contexto:

- **Sujeto:** El elemento principal (ej. "Genera un dragón dorado").
- **Estilo:** El medio artístico deseado (ej. "estilo óleo, con pinceladas impresionistas").
- **Contexto:** La escena o atmósfera (ej. "volando sobre un castillo en ruinas al atardecer, luz dramática").

Prompt para programación

- Asistencia en Debugging
- Explicación de Código
- Generación de documentación y comentarios
- Traducción entre lenguajes de programación
- Refactorización del código
- Optimización de rendimiento
- Identificación de vulnerabilidades
- Generación de scripts de SQL

Prompt para aprender idiomas

Actua como profesor de catalán de conversacion

Hazme preguntas para practicar conversación hablada, de a una por vez y espera mi respuesta

Para terminar la conversacion espera la frase TERMINAR LECCION

Actua com a dependent/a d'una fleca (forn de pa) a Barcelona.

Fes-me preguntes sobre què vull comprar. Jo sóc el client.

Per acabar la conversa, espera la frase ACABAR LLIÇÓ.

Ejercicio

- En tu LLM favorito prueba 3 prompts distintos



Definición

LangChain es un **framework de software** diseñado para facilitar el desarrollo de aplicaciones impulsadas por grandes modelos de lenguaje (**LLMs**) como GPT-4, Llama, y otros. Actúa como una **capa de abstracción** que permite a los desarrolladores conectar estos modelos de lenguaje con **aplicaciones**.

Aplicaciones:

- **Chatbots Contextuales:** Que pueden responder preguntas basándose en la documentación o datos internos de una empresa.
- **Sistemas de Preguntas y Respuestas (Q&A):** Donde el modelo consulta fuentes de datos específicas (índices) para generar respuestas precisas.
- **Agentes de Automatización:** Capaces de interactuar con APIs y herramientas de terceros para realizar acciones (como agendar una reunión o generar un reporte).

Hola mundo con la API Gemini + LangChain

- [Google AI Studio](#)
- Crear API KEY
- `$env:GOOGLE_API_KEY="-----"`
- crear carpeta `langchain_bootcamp`
- en la carpeta: `python -m venv venv1` (para crear virtual environment)
- en `/venv1/scripts/actíivate.ps1` (ejecutar para activarlo)
- `pip install langchain langchain-google-genai`
- en VS seleccionar el interprete de python en el `venv/scripts/python.exe` (con CTRL + SHIFT + P)

Hola mundo con la API Gemini + LangChain

```
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.7)

pregunta = input("¿Pregunta? :")
respuesta = llm.invoke(pregunta)
print("Respuesta: ", respuesta.content)
```

LLMs en local



Es una **plataforma de código abierto y una herramienta de línea de comandos** que simplifica drásticamente la ejecución de modelos de lenguaje grandes (LLMs) de código abierto, como Llama 3 o Mistral, **directamente en tu propio ordenador local** (sea Windows, Mac o Linux).

Hola mundo con Llama en local

- Descargar desde <https://ollama.com/>
- `ollama pull tinyllama`
- `ollama list`
- `pip install langchain-ollama`
- Ejecutar el Ollama para que arranque el servidor

Hola mundo con Llama en local

```
from langchain_ollama.llms import OllamaLLM
```

```
llm = OllamaLLM(model="tinyllama")
```

```
pregunta = input("What can I do for you? :")
```

```
respuesta = llm.invoke(pregunta)
```

```
print("Answer: ", respuesta)
```

Usando plantillas de prompts

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import PromptTemplate

chat = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.7)

plantilla = PromptTemplate(
    input_variables=["nombre"],
    template="Saluda al usuario con su nombre.\nNombre del usuario: {nombre} \n\nAsistente:"
)

#Encadenamos operaciones con | (operador pipe o tubería)
chain = plantilla | chat

resultado = chain.invoke({"nombre": "Jorge"})
print(resultado.content)
```



Qué es Streamlit

Streamlit es un **framework de Python de código abierto** diseñado para que los científicos de datos e ingenieros de *Machine Learning* puedan **crear y compartir aplicaciones web interactivas** de forma rápida y sencilla, utilizando **solo código Python puro**, sin necesidad de conocimientos de *frontend* (HTML, CSS, JavaScript).

[Mirar la App Gallery](#)

Funcionamiento

- **Modelo de Ejecución: El Rerun Completo**

- Principio Clave: El script de Python se **re-ejecuta completamente** de principio a fin con cada interacción del usuario (clic, cambio de slider, etc.).
- Implicación: Las variables locales se restablecen.

- **Persistencia del Estado**

- **st.session_state**: Único mecanismo para preservar el estado (variables) a través de los reruns. Se comporta como un diccionario en la memoria de la sesión.
- Sincronización: Los widgets se vinculan a `session_state` usando el parámetro `key` para manejar el estado automáticamente.

Chatbot con Streamlit, paso 1

- pip install streamlit

```
import streamlit as st
from langchain_google_genai import ChatGoogleGenerativeAI

# Configuración inicial
st.set_page_config(page_title="Chatbot Básico", page_icon="🤖")
st.title("🤖 Chatbot Básico con LangChain")
st.markdown("Este es un *chatbot de ejemplo* construido con LangChain + Streamlit.")

chat_model = ChatGoogleGenerativeAI(model="gemini-2.5-flash")
```

- streamlit run streamlit_paso1.py

Chatbot con Streamlit, paso 2

- descargar streamlite_paso2.py de Discord
- `streamlit run streamlit_paso2.py`

Streamlit Community Cloud

- Registrarse
- Crear aplicación
- Conectar al repositorio de github
 - crear un requirements.txt con las librerías necesarias

Ejercicio

- Añadir un menú a la derecha con la configuración del modelo:
 - Un slider para la “temperatura”
 - Un select para intercambiar el modelo
- Añadir un botón para limpiar la conversación